

Configuración de un servidor web seguro



Configuración de un servidor web seguro

Master en IoT

Objetivo

Aprender a configurar un servidor web seguro.

Índice

- 1) Configuración básica de un servidor web tipo LAMP
 - a) Instalación y configuración de Apache
- 2) Configuración segura de un servidor web
 - b) Habilitación de HTTPS
 - c) Generación de un certificado para el servidor
- 3) Auditoría de la seguridad del servidor HTTPS

Calificación

La calificación está indicada en los apartados. Debéis llamar a la profesora cuando terminéis cada bloque, para que lo anote.

Nombres y apellidos
Gissela Patricia Tasiguano
Ariel Gámez

No olvidéis guardar una copia de las MVs una vez terminada la sesión, para continuar con la práctica en la sesión de mañana

Configuración de la red

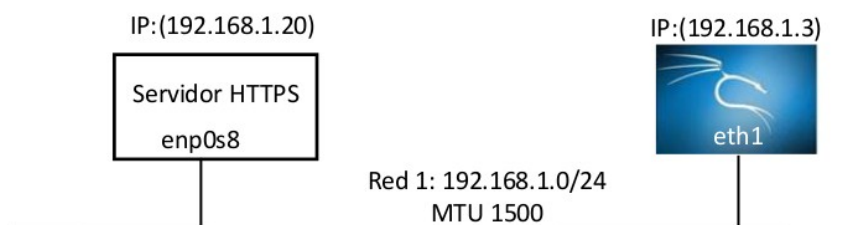


Figura 1

Vamos a usar la estructura de red de la Figura 1. Para ello, realizaremos 1 clonación de la máquina virtual `rys23_2` (usuario/usuario: `rys`): `SERV` (el servidor HTTPS) → recuerda generar nuevas direcciones MAC y realizar clonación enlazada. En la máquina `SERV` montaremos nuestro servidor LAMP. También utilizaremos la máquina Kali (`kali/kali`) como máquina de auditoría, para comprobar si el servidor funciona correctamente y realizar algunas comprobaciones de seguridad.

Clonar la MV Kali antes de empezar. Recordad comprobar su conexión a internet antes de modificar los adaptadores de red.

Una vez clonada la máquina rys23_2, comprueba que tiene conexión a Internet. Cierra la MV y después habilita los adaptadores a la red correspondiente (adaptador 1 conectado a NAT y el adaptador 2 conectado a la red interna Red 1).

Nota acerca del teclado en la Mv Kali: el teclado por defecto es inglés. Si quieres pasar a teclado español:

- ir a settings
- descativar el “default”
- añadir teclado español

A continuación, vamos a describir los pasos a seguir para configurar nuestro entorno de trabajo:

1. La distribución de Debian que estamos utilizando ya trae instalado el SW para un servidor apache2. Instala el software de php y el cortafuegos UFW (Uncomplicated Firewall):

```
$ sudo apt-get update
$ sudo apt-get install firewalld
$ sudo apt-get install php php-common (opcional)
```

Comprueba el estado del firewall y desactívalo de momento. Para comprobar que se ha instalado correctamente puedes utilizar:

```
$ sudo dpkg - -list
```

Para pararlo:

```
$ sudo systemctl stop firewalld
```

2. En la máquina de Kali instala la herramienta ssltest.sh:

```
$ git clone --depth 1 https://github.com/testssl/testssl.sh.git
```

3. Comprueba la correcta configuración persistente de red de las dos MVs y su conectividad (recuerda que los nombres de interfaces varían con las distribuciones de Linux).

Parte 1 : Configuración del servidor LAMP

Se denomina "LAMP" a un servidor que se instala sobre una plataforma Linux para crear un servidor que aloje sitios y aplicaciones web dinámicas. Este término en realidad es un acrónimo que representa un sistema operativo Linux con un servidor Apache. Los datos del sitio web son almacenados en una base de datos MySQL y el contenido dinámico es procesado con PHP.

LAMP = **L**inux + **A**pache + **M**ySQL + **P**HP

Ejercicio 1: Configuración del servidor Apache.

Lo primero que vamos a hacer es comprobar la versión de apache 2 que hemos instalado.

1. En la máquina Servidor HTTPS haz:

```
$ man apache2
```

Apunta las opciones que te parezcan interesantes:

-t (Syntax Test): Comprueba que no hay errores de sintaxis en los archivos de configuración. Se utiliza siempre antes de reiniciar para evitar que el servidor se caiga por un error tipográfico.

-S (Show Settings): Muestra la configuración de los VirtualHosts. Muy útil para verificar rápidamente qué puertos (como el 443 para HTTPS) están respondiendo.

-M (Dump Modules): Lista los módulos activos. Clave para reducir la superficie de ataque (desactivando lo que no utilizamos) y confirmar que el módulo ssl está cargado.

-v / -V (Version): Revela la versión exacta de Apache. Primer paso indispensable para buscar vulnerabilidades conocidas (CVEs) en nuestro servidor.

-X (Debug Mode): Arranca el servidor en modo depuración. Ideal para diagnosticar problemas o bloqueos extraños en tiempo real.

2. Comprueba qué versión de apache2 está instalada:

```
$ sudo apache2 -V
```

```
Server version: Apache/2.4.66 (Debian)
```

Todos los ficheros necesarios para que este servidor se ejecute están almacenados en /etc/apache2.

3. Lista el contenido de este directorio. ¿Cómo se llama el principal fichero de configuración de apache2? apache2.conf

4. Abre este fichero y consulta la jerarquía de configuración que aparece descrita en él.

En ese árbol, ¿cuántos directorios hay? Cuatro exactamente: /etc/apache2/ (El directorio principal base), mods-enabled/ (Para los módulos), conf-enabled/ (Para fragmentos de configuración global) y sites-enabled/ (Para las configuraciones de los VirtualHosts o sitios web)

5. Lee el archivo ports.conf y contesta a estas preguntas:

¿En qué puerto está escuchando el servidor por defecto? Puerto 80 (Listen 80)

¿Qué protocolo de la capa de aplicación utiliza por defecto este servidor web? HTTP (Hypertext Transfer Protocol), el protocolo estándar de la capa de aplicación asociado internacionalmente al puerto 80

¿Cuándo estará el servidor escuchando por el puerto 443? El servidor solo escuchará por el puerto 443 (el puerto estándar para HTTPS) si se cumple la condición de tener cargados los módulos de seguridad.

6. Comprueba si el servidor está correctamente configurado:

```
$ sudo apache2ctl configtest
```

¿Qué warning aparece?

```
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

¿Cómo dice que se puede solucionar?

Apache está diseñado para alojar sitios web con nombres de dominio (como www.servidorweb.com). Al arrancar, intenta averiguar cuál es su propio "Nombre de Dominio Completamente Calificado" (FQDN). Como no le hemos configurado ninguno explícitamente a nivel global, se queja un poco y decide usar la IP local (127.0.1.1 o localhost) por defecto.

Solución: Crear un archivo de configuración global en el directorio /etc/apache2/

1. Crear el archivo de configuración con el nombre del servidor (usaremos localhost para tu entorno de pruebas):

```
echo "ServerName localhost" | sudo tee /etc/apache2/conf-available/servername.conf
```

2. Habilitar esta nueva configuración:

```
sudo a2enconf servername
```

3. Recargar el servicio de Apache para aplicar los cambios:

```
sudo systemctl reload apache2
```

7. En el fichero `apache2.conf` indica cuál va a ser el nombre por el que se va a conocer nuestro servidor LAMP:

ServerName **www.miserv.com**

Si se fueran a realizar cambios más complejos en el fichero de configuración conviene primero copiar el fichero original (en `apache2.0.conf`, por ejemplo) y luego ya empezar a hacer modificaciones sobre el fichero `apache2.conf`. De esa manera si alguna modificación no es correcta y se desconfigura por completo el servidor siempre podremos restablecer rápidamente la configuración original sin prácticamente afectar al servicio.

8. Comprueba que el servidor está correctamente configurado:

```
$ sudo apache2ctl configtest
```

¿Qué aparece ahora? **Syntax OK**

9. Como nuestro Servidor no consta en el registro de dominios DNS y no vamos a realizar una consulta DNS, necesitaremos resolver la url de forma local. Como vamos a conectarnos al servidor web desde la máquina Kali, en el fichero `/etc/hosts` de esta máquina debemos indicar que la dirección `192.168.1.20` se corresponde con **www.miserv.com**. Para ello añadimos en este fichero la siguiente línea:

```
192.168.1.20 www.miserv.com
```

10. En la MV Servidor, reiniciamos el servidor (se ha arrancado por defecto una vez que los hemos instalado):

```
$ sudo systemctl restart apache2
```

Utiliza el comando `netstat -tan` para comprobar que el servidor está arrancado y escuchando en el puerto que le corresponde (copia la línea que indica que el servidor está arrancado):

```
tcp6    0      0 :::80          :::* LISTEN
```

:80 Es el puerto estándar del protocolo HTTP. Nos indica que el servidor web Apache está efectivamente arrancado.

::: Significa que está escuchando peticiones en todas las interfaces de red disponibles (Apache suele mapear las conexiones IPv4 a través de la interfaz IPv6 por defecto).

LISTEN Confirma que el estado del puerto es "Escuchando", es decir, está abierto y esperando a que alguien (como tu máquina Kali) se conecte a él.

11. Abre el navegador de Kali y comprueba que eres capaz de acceder a `http://www.miserv.com`. Si todo se ha configurado correctamente el navegador te ofrecerá la página web en html que el servidor trae por defecto. ¿Es segura esta conexión? ¿Por qué?

Cuando se intentó conectar a <http://www.miserv.com> mediante la mv de Kali se obtuvo error. Se realizaron las siguientes comprobaciones:

- Resolución de DNS local:

```
echo "192.168.1.20 www.miserv.com" | sudo tee -a /etc/hosts
```
- Verificación DNS:

```
ping -c 4 www.miserv.com
```

- Verificación de acceso: `curl -I http://www.miserv.com`
- Verificación de conexión de red interna Red1 por las dos mv: `ip addr show eth1 |`
- `ip addr show enp0s8`

Se llegó a la conclusión que el problema era del cortafuegos. Para solucionarlo se llevó a cabo las siguientes modificaciones en el mv de SERV:

1. Permite el tráfico HTTP (puerto 80)

```
sudo firewall-cmd --add-service=http --permanent
```

2. Permite el tráfico HTTPS (puerto 443)

```
sudo firewall-cmd --add-service=https --permanent
```

3. Recarga para aplicar los cambios

```
sudo firewall-cmd --reload
```

Para responder si la conexión es segura y por qué. **No, no es segura.** Se basa en HTTP (puerto 80), lo que significa que los datos viajan en texto plano sin cifrado. Cualquier persona en la misma red podría interceptar o manipular la información.

Ejercicio 2: Configuración de php en el servidor

La configuración por defecto del servidor permite que este funcione como un servidor web estático, pero si queremos que nuestro servidor tenga más funcionalidades, es necesario instalar otros programas, como puede ser un gestor de base de datos como mysql, un sistema de gestión de contenidos tipo wordpress o un lenguaje de programación como php, que permite crear páginas web con contenido dinámico (páginas que no sean simple código HTML).

En esta práctica vamos a añadir la funcionalidad de php, para lo que seguiremos los siguientes pasos:

1. Ve al directorio `/var/www/html` y cambia el nombre del fichero `index.html` por `iold.html` (`sudo mv index.html iold.html`). En este directorio crea un fichero llamado `info.php` con el siguiente contenido para generar una página web que ejecute php:

```
<?php
phpinfo();
?>
```

2. Modifica la configuración del servidor para indicarle que la nueva página de inicio es la creada en el punto 2, para ello busca el fichero de configuración correspondiente en el directorio `/etc/apache2/mods-enabled`. Apunta el nombre del directorio y el cambio realizado.

Directorio: El enunciado solicita el "director", pero se refiere al archivo de configuración concreto dentro de ese directorio `/etc/apache2/mods-enabled/dir.conf`

Cambio: Agregar el nombre del archivo `info.php` al inicio.

```
<IfModule mod_dir.c>
    DirectoryIndex info.php index.html index.cgi index.pl index.php index.xhtml
index.htm
</IfModule>
```

Nota: Si no has conseguido encontrar el fichero de configuración donde se debe realizar el cambio puedes probar que todo funciona correctamente utilizando la dirección web `http://www.miserv.com/info.php` en el navegador de Kali.

3. Reinicia apache y comprueba los resultados conectándote desde el navegador de Kali a `http://www.miserv2.com` (antes de conectarte borra el historial de navegación).

```
sudo systemctl restart apache2
```

4. Vemos que php está correctamente instalado. Sin embargo, la página cargada ofrece información sobre el servidor, lo que puede ser peligroso para su seguridad.

Cualquier ejecución de código o consulta sobre el servidor puede considerarse una posible vulnerabilidad, por lo que debemos evitar la ejecución de php desde un cliente. Para ello:

a) edita el fichero de configuración /etc/apache2/mods-enabled

```
ls /etc/apache2/mods-enabled/ | grep php  
php7.4.conf  
php7.4.load
```

b) indica los tipos de módulos permitidos

\$ sudo a2dismod + vers php (la versión coincide con el nombre del fichero de configuración de php del directorio /etc/apache2/mods-enabled)

```
sudo a2dismod php7.4
```

c) reinicia el servidor apache.

```
sudo systemctl restart apache2
```

A continuación, y después de borrar el historial de navegación accede desde Kali a

http://www.miserv.com si has conseguido modificar la página de inicio o a

http://www.miserv.com/info.php si no lo has conseguido. ¿dónde se pide que modifiquen la página de inicio y cómo saben cuál es?. Comprueba que NO se puede ejecutar el código php.

En el punto 2 se ha modificado la página de inicio. Como Apache ya no sabe cómo "traducir" ni ejecutar el código PHP, comprobamos que NO se ejecuta. Confirmando así que hemos mitigado la vulnerabilidad con éxito.

```
sudo nano /etc/apache2/mods-enabled/dir.conf
```

```
<IfModule mod_dir.c>  
    DirectoryIndex index.html index.cgi index.pl index.php index.xhtml index.htm  
</IfModule>
```

5. Por último, vamos a recuperar como página de inicio la que estaba por defecto. Ve al directorio /var/www/html y cambia el nombre del fichero iold.html por index.html.

```
sudo mv iold.html index.html
```

— Hasta aquí primer bloque: 3 p ---

SEGURIDAD

Una vez comprobado que tenemos 'php' funcionando correctamente conviene borrar este archivo info.php, ya que contiene información relevante del servidor que podría ser leída por usuarios NO AUTORIZADOS.

```
sudo rm /var/www/html/info.php
```

Parte 2: Configuración de un servidor web seguro: habilitar HTTPS

La navegación web segura (https) utiliza el protocolo SSL/TLS. Este protocolo se basa en el uso de certificados, que solamente son útiles si una autoridad certificadora los ha validado. Los certificados válidos son necesarios para configurar un servidor web seguro, ya que permiten al cliente confiar en que se está conectando al sitio que dice ser.

Normalmente el certificado lo debemos enviar a una autoridad certificadora para que lo firme, pero para realizar la práctica, por razones de coste en tiempo y dinero, vamos a firmarnos nuestro

certificado, lo que evidentemente lo invalida y como veremos más adelante, hace que el navegador desconfíe de nuestro servidor, avisando al cliente de que el servidor al que se quiere conectar no tiene un certificado válido.

Ejercicio 3. Creación de un certificado para el servidor.

Vamos a utilizar una utilidad de openssl que permite crear los certificados y autofirmarlos, además de guardarlos directamente en los directorios adecuados. Debéis revisar la teoría sobre certificados para entender todos los pasos que sigue el proceso de certificación y poder responder correctamente a las preguntas de este apartado.

```
$ sudo openssl req -x509 -nodes -newkey rsa:2048 -keyout /etc/ssl/private/servkey.pem -out /etc/ssl/certs/servcert.pem
```

Añadir los datos para el certificado

- Country Name: **SP**
- State Name: **Madrid**
- Locality Name: **Madrid**
- Organization Name: **CSV / MíoT**
- Organization Unit Name:
- Common Name: **www.miserv.com**
- Email Address: **estudiante@ucm.es**

Generación del certificado y la clave privada:

¿Dónde se guarda la clave privada?

Se guarda en la ruta `/etc/ssl/private/servkey.pem`. Esto se lo hemos indicado al comando mediante el parámetro `-keyout`. Por seguridad este directorio suele tener permisos muy restrictivos. Se guarda la clave privada.

¿Dónde se guarda el certificado?

Se guarda en la ruta `/etc/ssl/certs/servcert.pem`. Esto viene definido por el parámetro `-out`. Esta es la clave pública que se envían a los clientes.

¿Qué tamaño tiene la clave?:

El tamaño es de 2048 bits. Lo hemos especificado con el argumento `rsa:2048`, lo que indica que se genera una clave utilizando el algoritmo RSA con esa longitud.

Vamos a comprobar ahora que todo se ha creado correctamente, comprueba que existen los siguientes ficheros:

- `/etc/ssl/certs/servcert.pem`

```
ls -l /etc/ssl/certs/servcert.pem
-rw-r--r-- 1 root root 1399 mar 30 17:09 /etc/ssl/certs/servcert.pem
```

Permisos `-rw-r--r--` (644) lectura pública: El dueño (`root`) puede leer y escribir, pero cualquier usuario del sistema puede leerlo.

- `/etc/ssl/private/servkey.pem`

```
su -root
ls -l /etc/ssl/private/servkey.pem
-rw----- 1 root root 1704 mar 30 17:08 /etc/ssl/private/servkey.pem
```


Permisos -rw----- (600) restricción total Solo el dueño (root) tiene permiso para leer o escribir. Nadie más en el sistema puede siquiera ver su contenido.

Para ver el contenido del certificado:

```
$ openssl x509 -in servcert.pem -noout -text
```

Comprueba los datos del certificado y los algoritmos de cifrado y hash:

```
sudo openssl x509 -in /etc/ssl/certs/servcert.pem -noout -text
[sudo] password for usuario:
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      1b:f5:67:dd:41:9f:40:11:47:53:ac:4b:fd:33:ab:dd:09:fe:b8:c0
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = SP, ST = Madrid, L = Madrid, O = CSV/MioT, CN =
www.miserv.com, emailAddress = estudiante@ucm.es
    Validity
      Not Before: Mar 30 15:09:52 2026 GMT
      Not After : Apr 29 15:09:52 2026 GMT
    Subject: C = SP, ST = Madrid, L = Madrid, O = CSV/MioT, CN =
www.miserv.com, emailAddress = estudiante@ucm.es
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (2048 bit)
      Modulus:
        00:ab:43:b5:c6:1a:2b:ea:0f:86:8b:69:ce:cc:a5:
        e2:c6:9e:fd:a3:07:b8:e0:3d:75:1f:9b:42:fc:50:
        75:89:36:06:b1:41:fa:e8:f7:74:7b:8c:d1:4d:27:
        59:4b:9e:a9:98:ae:63:30:be:1d:0c:94:a0:c5:a8:
        36:bc:da:6e:45:f2:08:c8:f5:7b:d5:09:af:43:4a:
        13:c4:34:66:24:d5:c6:34:8a:74:c5:10:9e:0b:67:
        2c:6a:93:39:83:b7:6a:7c:eb:93:83:c1:1f:70:bb:
        31:a6:5a:8c:22:bb:37:c3:fb:c9:cb:2e:57:f7:97:
        3c:c6:94:12:ec:9e:4b:6a:f1:e7:ef:88:cf:fa:5a:
        0c:d5:a8:11:2a:bd:e9:1c:ea:2f:be:db:7f:94:b1:
        93:66:66:7f:cf:f3:76:df:61:db:57:f3:2b:1a:c6:
        3d:19:2a:c3:59:71:26:9a:71:7a:b3:db:10:4a:45:
        a1:b5:f3:ed:31:3b:5f:bf:27:95:7f:4d:0c:f0:78:
        34:f1:d7:13:6f:d4:bf:23:88:6a:80:ee:be:5c:4b:
        33:7e:e9:53:b6:f0:90:fc:dd:c9:8a:a3:a8:94:50:
        35:e4:52:27:9d:82:d7:66:77:f8:12:75:e0:d1:21:
        0b:eb:bc:00:f5:ce:cf:62:2f:cd:c6:f8:9a:ff:8a:
        53:b9
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        A3:58:C3:AD:2E:9E:B6:A9:49:29:8F:2A:47:2A:C9:9B:F3:6A:DD:E8
      X509v3 Authority Key Identifier:
        keyid:A3:58:C3:AD:2E:9E:B6:A9:49:29:8F:2A:47:2A:C9:9B:F3:6A:DD:E8

      X509v3 Basic Constraints: critical
      CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
      a2:07:3d:8c:c4:d6:c5:0d:34:11:20:db:70:c9:de:89:6c:91:
      b8:af:ed:86:da:fb:97:2a:52:7f:d7:7f:58:e1:51:79:03:84:
      af:30:2b:ff:08:0a:2f:82:cf:2b:59:bf:b4:85:4f:20:49:18:
      93:c1:b5:96:39:ed:be:8d:7d:0a:06:d0:40:9a:50:1b:d6:cc:
      62:d1:33:01:cf:4a:5e:bb:b2:c8:02:5c:f4:9b:a1:cb:7e:d9:
      11:8d:48:42:f6:76:62:31:8a:55:8e:7f:58:5c:e8:31:17:6a:
      44:0d:0e:48:61:ae:41:77:e4:ed:02:a7:0d:26:a0:1c:b1:ee:
      d8:0c:e2:65:0f:d4:5e:9c:ac:6d:d1:5f:d1:6d:cf:09:b5:17:
      7d:36:b6:4d:40:00:10:45:05:45:e0:b5:e9:38:e9:96:e5:f3:
```

```
64:5c:8f:7b:96:60:74:bc:a6:67:6e:f8:a2:e2:a1:7e:99:63:
ad:00:c9:ac:9f:a4:c8:fc:30:7f:a8:33:00:33:eb:d1:66:1d:
38:9f:f9:fa:c2:a9:59:0e:68:36:f9:23:a0:50:48:d2:77:61:
e4:d3:b4:a8:8a:0b:f8:96:9a:da:e0:0a:55:16:98:b8:ad:a5:
71:3c:60:0d:9f:aa:b8:ca:25:0a:14:39:3f:53:a7:15:7c:ac:
36:a7:e8:a5
```

Datos del certificado:

- Emisor (Issuer) y Sujeto (Subject): `C = SP, ST = Madrid, L = Madrid, O = CSV/MioT, CN = www.miserv.com, emailAddress = estudiante@ucm.es` (Como es un certificado autofirmado, el Emisor y el Sujeto son exactamente los mismos. Nosotros mismo nos damos fe de quiénes somos).

- Validez (Validity): Válido desde el 30 de marzo de 2026 hasta el 29 de abril de 2026 (tiene una validez de 30 días, que es el valor por defecto de OpenSSL si no le indicamos lo contrario con el parámetro `-days`).

Algoritmos:

- Algoritmo de Hash (Firma): `sha256WithRSAEncryption`. Esto significa que utiliza la función hash SHA-256 para garantizar que el certificado no ha sido alterado.

- Algoritmo de Cifrado (Clave Pública): `rsaEncryption`.

- Tamaño de la clave: `RSA Public-Key: (2048 bit)`. Esta es la cerradura pública de 2048 bits que se enviará a los clientes para que cifren los datos antes de enviárnolos.

El certificado que nuestro servidor va a “ofrecer al exterior” para indicar que es un servidor confiable y que la clave pública es la correcta es **servcert.pem**

Ejercicio 4. Completar la configuración https

Ya tenemos el certificado (autofirmado), pero nos falta activar el módulo ssl para que nuestro servidor escuche en el puerto 443. Si ejecutas:

```
$ netstat -tan
```

verás que sigue escuchando en el puerto 80.

```
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:8888            0.0.0.0:*               LISTEN
tcp6       0      0 :::80                  :::*                    LISTEN
tcp6       0      0 :::22                  :::*                    LISTEN
tcp6       0      0 :::1:631                :::*                    LISTEN
tcp6       0      0 :::8888                 :::*                    LISTEN
```

para habilitar TLS/SSL en nuestro servidor:

1. Habilita/activa el módulo de ssl del servidor Apache.

Debemos ahora activar el módulo `mod_ssl` definido en `/etc/apache2/mods-available/ssl.conf` y `/etc/apache2/mods-available/ssl.load` con:

```
$ sudo a2enmod ssl
```

Este comando crea enlaces simbólicos a los ficheros anteriores en el directorio `/etc/apache2/mods-enabled`.

Reinicia Apache:

```
$ sudo systemctl restart apache2
```

2 Habilita el sitio con SSL definido en /etc/apache2/sites-available/default-ssl:

```
$ sudo a2ensite default-ssl
```

Este comando crea un enlace simbólico al fichero anterior en el directorio /etc/apache2/sites-enabled.

Recarga Apache:

```
$ sudo systemctl reload apache2
```

3. Para que se pueda producir la comunicación segura entre el servidor y el cliente es necesario que indiquemos a Apache donde está el certificado del servidor, firmado por la Autoridad (CA), y la clave.

Modifica el fichero /etc/apache2/sites-enabled/default-ssl.conf para indicar los directorios donde se podrán encontrar el certificado y la clave del servidor. Busca estas líneas y cambia la información por el path real de nuestro certificado y nuestra clave:

```
SSLCertificateFile /etc/ssl/certs/www.miserv.com.crt
```

```
SSLCertificateKeyFile /etc/ssl/private/www.miserv.com.key
```

4. Recarga y reinicia la configuración de Apache (si nos pregunta introducir la contraseña que se dio al crear el servercsr.pem):

```
$ sudo systemctl reload apache2
```

```
$ sudo systemctl restart apache2
```

Ejercicio 5: Comprobar que la conexión con el servidor web es segura.

1. Accede a https://www.miserv.com desde el navegador de Kali.

¿Qué warning devuelve y por qué?

```
- Warning: Potential Security Risk Ahead
Firefox detected a potential security threat and did not continue to
www.miserv.com. If you visit this site, attackers could try to steal
information like your passwords, emails, or credit card details.
```

Ese mensaje aparece porque Firefox ha detectado un problema crítico con el certificado de seguridad (SSL/TLS). Debido a que es un certificado autofirmado probablemente para el sitio web www.miserv.com. Firefox actúa como un "notario" digital y, al no poder verificar que la conexión es privada y auténtica, bloquea el acceso para proteger nuestros datos.

2. Pulsa en Advanced.

¿Qué aviso da ahora?

```
www.miserv.com uses an invalid security certificate.
The certificate is not trusted because it is self-signed.
Error code: MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT
View Certificate
```

¿Te deja conectarte?

Sí, el navegador nos permite conectarnos si pulsamos el botón de la derecha: **Accept the Risk and Continue** (Aceptar el riesgo y continuar).

Desde el navegador, comprueba la información del certificado de tu servidor y las propiedades de la conexión pulsando sobre el candado que aparece delante de la URL. Describe las propiedades de la conexión (versión protocolo TLS y algoritmos criptográficos usados).

- Algoritmo de Clave Pública: RSA con una longitud de 2048 bits. Es una longitud robusta y segura para el intercambio de claves.

- Protocolo TLS: La conexión se está negociando bajo TLS 1.3, que son las versiones que soportan algoritmos de cifrado modernos.

Connection Encrypted (TLS_AES_128_GCM_SHA256, 128 bit keys, TLS 1.3)

- Cifrado (Cipher Suite): Basado en el certificado, el servidor utiliza una combinación de AES-256 o AES-128 para el cifrado de datos y ECDHE (Elliptic Curve Diffie-Hellman) para el intercambio de claves, garantizando "Perfect Forward Secrecy".

¿En qué puertos está escuchando ahora el servidor?

```
netstat -tan
```

```
Active Internet connections (servers and established)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp6	0	0	:::80	:::*	LISTEN

```
...
```

tcp6	0	0	:::443	:::*	LISTEN
------	---	---	--------	------	--------

Puerto 443 (TCP): Es el puerto principal que está utilizando ahora mismo. Es el estándar para conexiones HTTPS.

Puerto 80 (TCP): Aunque no lo vemos directamente en la captura de "Detalles Técnicos", los servidores web suelen tener este puerto abierto para escuchar peticiones HTTP estándar.

Ejercicio 6. Configuración del firewall para que solamente permita conexiones https.

Activad el firewall y probad a conectaros ahora con el servidor desde Kali. ¿Por qué ahora ya no podéis hacerlo ni por http ni por https?

```
# 1. Denegar el tráfico HTTP (puerto 80)
```

```
sudo firewall-cmd --remove-service=http --permanent
```

```
# 2. Denegar el tráfico HTTPS (puerto 443)
```

```
sudo firewall-cmd --remove-service=https --permanent
```

```
# 3. Recarga para aplicar los cambios
```

```
sudo firewall-cmd --reload
```

Al activar un firewall en Linux sin reglas previas, este aplica una política de bloqueo total por defecto. Como resultado, intercepta y descarta todo el tráfico entrante, incluyendo las peticiones a los puertos 80 (HTTP) y 443 (HTTPS), impidiendo que lleguen al servidor web. En nuestro caso lo hemos tenido que hacer manual como se describe arriba.

Consultad las zonas activas del firewall. Asignadle la zona *public* y consultad la configuración por defecto de dicha zona. ¿Qué cambios debes realizar en el firewall para que permita la conexión https pero no la http?.

```
# 1. Consultad las zonas activas del firewall
```

```
sudo firewall-cmd --get-active-zones
```

```
public
```

```
interfaces: enp0s3
```

```
# 2. Asignadle la zona public y consultad la configuración por defecto
```

```
sudo firewall-cmd --set-default-zone=public
```

```
Warning: ZONE_ALREADY_SET: public
```

```
success
```

```
# 3. Consultar la configuración por defecto de esa zona
```

```
sudo firewall-cmd --zone=public --list-all
```

```
public (active)
```

```
target: default
```

```
icmp-block-inversion: no
interfaces: enp0s3
sources:
services: dhcpv6-client ssh
ports:
protocols:
forward: no
masquerade: no
forward-ports:
source-ports:
icmp-blocks:
rich rules:
```

4. Los comandos para realizar estos cambios en tu servidor son:

```
# 1. Añadir HTTPS a la zona public
sudo firewall-cmd --zone=public --add-service=https --permanent
# 2. Asegurarnos de quitar HTTP (por si lo habíamos abierto antes)
sudo firewall-cmd --zone=public --remove-service=http --permanent
# 3. Recargar el firewall para aplicar el bloqueo y el permiso
sudo firewall-cmd --reload
```

Realizad los cambios y comprobad que ahora sí podéis conectar con el servidor a través de https. Probad también a conectaros por http. ¿Qué sucede?.

```
(kali㉿kali)-[~]
└─$ curl -I https://www.miserv.com
curl: (60) SSL certificate problem: self signed certificate
More details here: https://curl.se/docs/sslcerts.html
```

curl failed to verify the legitimacy of the server and therefore could not establish a secure connection to it. To learn more about this situation and how to fix it, please visit the web page mentioned above.

Este mensaje significa que curl no puede confiar en la conexión segura porque el servidor utiliza el certificado autofirmado. Pero si puede conectarse.

```
(kali㉿kali)-[~]
└─$ curl -I http://www.miserv.com
curl: (7) Failed to connect to www.miserv.com port 80 after 2 ms: No route to host
```

La máquina Kali no puede encontrar un camino para llegar al servidor. A diferencia del error anterior (que era un problema de seguridad SSL).

— Hasta aquí segundo bloque: 4 p ---

Parte 3: Auditar y mejorar la seguridad del servidor.

Ahora vamos a utilizar la herramienta de auditoría del protocolo TLS/SSL que hemos descargado al principio:

```
$ ./testssl.sh/testssl.sh 192.168.1.20
```

Según la información mostrada por la auditoría de seguridad:

```
# 1. Acceder
cd testssl.sh
# 2. Volver a lanzar la auditoría
./testssl.sh 192.168.1.20
```

¿Qué versiones del protocolo TLS se permiten por defecto?

Se están permitiendo TLS 1.0, TLS 1.1, TLS 1.2 y TLS 1.3. Esto se ve en la sección `Testing protocols`, donde TLS 1 y 1.1 aparecen marcados como "deprecated" (obsoletos).

¿Qué tamaños de clave para el algoritmo de cifrado simétrico?

Se permiten tamaños de clave de 128 bits y 256 bits. En la lista de cifrados (`Testing server's cipher preferences`) vemos constantemente referencias a algoritmos como AES y Camellia seguidos de 128 o 256.

¿Qué algoritmos de función resumen (hash)?

Se están utilizando principalmente SHA (que equivale a SHA-1), SHA256 y SHA384. (También se ofrecen SHA224 y SHA512 para las firmas, según la sección `sig_algs`).

¿Quién establece la preferencia a la hora de elegir los algoritmos de cifrado?

La preferencia la establece el cliente (el navegador web). El reporte indica claramente: `Has server cipher order? no (NOT ok)`, lo que significa que el servidor no impone su propio orden de seguridad, dejando que el cliente decida.

¿Ha intentado la herramienta conectarse a través del puerto 80? ¿Que dice al respecto?

Sí, lo ha intentado (por ejemplo, al comprobar la vulnerabilidad Opossum), pero la herramienta dice: `connection to port 80 failed`. Esto confirma que las reglas de `firewall` que aplicaste anteriormente están funcionando a la perfección y bloqueando el tráfico HTTP.

¿Ha realizado alguna verificación del certificado del servidor? ¿Hay algo en el certificado que lo arque como peligroso?

Sí, en la sección `Server defaults`. Lo marca como peligroso (`NOT ok`) por varias razones:

1. La cadena de confianza falla porque es un certificado autofirmado (`Chain of trust: NOT ok (self signed)`).
2. El nombre de dominio no coincide (`Trust: certificate does not match supplied URI`).
3. Le falta el atributo de nombre alternativo (`subjectAltName (SAN) missing`).

¿Ha encontrado vulnerabilidades conocidas? (¿cuáles?)

Sí, ha encontrado tres advertencias importantes en la sección `Testing vulnerabilities`:

1. BEAST (CVE-2011-3389): Aparece como `VULNERABLE` debido al uso de protocolos antiguos y cifrados débiles.
2. BREACH (CVE-2013-3587): Aparece como `potentially NOT ok` porque el servidor tiene activada la compresión HTTP ("gzip").
3. LUCKY13 (CVE-2013-0169): Aparece como `potentially VULNERABLE` por permitir cifrados de tipo CBC.

¿Ha obtenido un buen rating nuestro servidor? ¿Deberíamos mejorar su seguridad?

- No, ha obtenido una nota desastrosa: una "T" (la peor calificación en la escala de SSL Labs).
- Sí, es obligatorio mejorar la seguridad. El propio informe nos da las pistas para arreglarlo (`Grade cap reasons`): la nota ha bajado a "T" por culpa del certificado autofirmado, ha bajado a "M" por el desajuste del dominio, y ha bajado a "B" por seguir permitiendo los protocolos obsoletos TLS 1.0 y TLS 1.1.

A continuación, vamos a mejorar la seguridad del protocolo TLS instalado. Para ello, tenemos que modificar el fichero `/etc/apache2/mods-enabled/ssl.conf` de la máquina Servidor HTTPS. Vamos a permitir solamente los protocolos TLSv1.2 y TLSv1.3. Además, no vamos a permitir usar funciones resumen de tipo SHA o SHA1 ni el algoritmo RSA para intercambiar la clave simétrica, ya que no garantiza secreto perfecto hacia adelante.

```
sudo nano /etc/apache2/mods-enabled/ssl.conf
```

1. Deshabilita las versiones anteriores a TLSv1.2. Para hacerlo debes modificar la línea donde se definen las versiones soportadas en el fichero `ssl.conf`:

```
SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
```

2. Comprueba los algoritmos de cifrado permitidos por defecto en el fichero de configuración. Busca en el archivo de configuración `SSLCipherSuite` la línea:

```
SSLCipherSuite HIGH:!aNULL
```

El primer parámetro indica los algoritmos de cifrado permitidos. El parámetro `HIGH` indica que se aceptan solo algoritmos considerados robustos. El símbolo `!` significa no permitido, por lo que `!aNULL` significa que no se aceptan algoritmos sin autenticación.

¿Qué tendrías que añadir a esa línea para que no estuviesen permitidos los algoritmos SHA, SHA1 y RSA?. Modifica el fichero de configuración para que no se permitan estos algoritmos.

```
SSLCipherSuite HIGH:!aNULL:!SHA:!SHA1:!RSA
```

Con este cambio hemos logrado:

- `!SHA:!SHA1`: Prohíbe utilizar las funciones hash SHA y SHA-1 (que ya se consideran vulnerables a colisiones).
- `!RSA`: Prohíbe utilizar RSA para el intercambio de claves simétricas, forzando al servidor a usar algoritmos como Diffie-Hellman (DHE o ECDHE), que sí garantizan el Secreto Perfecto Hacia Adelante (Perfect Forward Secrecy).

3. Busca en el fichero la línea:

```
# SSLHonorCipherOrder On
```

Lee los comentarios asociados a esta línea. Por defecto, ¿qué puede ocurrir si esta línea está comentada?

```
# SSL server cipher order preference:
# Use server priorities for cipher algorithm choice.
# Clients may prefer lower grade encryption. You should enable this
# option if you want to enforce stronger encryption, and can afford
# the CPU cost, and did not override SSLCipherSuite in a way that puts
# insecure ciphers first.
# Default: Off
#SSLHonorCipherOrder on
```

Si la opción `SSLHonorCipherOrder` está comentada (Off por defecto), el servidor permite que los navegadores elijan algoritmos de cifrado débiles, comprometiendo la seguridad. Para forzar cifrados más fuertes y asegurar el servidor, se debe descomentar la línea para activar la opción `on`.

Vamos a mejorar esa política descomentando esta línea. ¿Quién establece ahora la preferencia a la hora de elegir los algoritmos de cifrado?

`SSLHonorCipherOrder On`

Al hacer esto, estamos imponiendo nuestra autoridad: le comunicamos a Apache que obligue a los clientes a utilizar estrictamente el orden de cifrados que nosotros hemos definido en el servidor (los más seguros primero), ignorando por completo las preferencias del navegador que se conecte.

4. Recarga y reinicia el servidor Apache:

```
$ sudo systemctl reload apache2
```

```
$ sudo systemctl restart apache2
```

5. Repite la auditoría desde la MV de Kali y apunta qué elementos de los marcados como peligrosos en la primera han desaparecido y cuáles no. ¿Cuál es la versión de TLS por defecto que va a utilizar el servidor?.

1. Peligros que HAN desaparecido (¡Objetivo cumplido!)

- Protocolos obsoletos: Ya no se ofrecen TLSv1.0 ni TLSv1.1. Solo TLS 1.2 y 1.3.

- Falta de autoridad del servidor: Ahora marca `Has server cipher order? yes (OK)`. El servidor ya impone el orden de los cifrados.

- Cifrados débiles: Han desaparecido los algoritmos vulnerables (SHA, SHA-1) y el intercambio de claves por RSA. Ahora se utiliza Secreto Perfecto Hacia Adelante (PFS).

2. Peligros que NO han desaparecido

Principalmente, los problemas relacionados con la identidad del certificado digital (que son normales en un entorno de prácticas):

- Certificado autofirmado: Al no estar emitido por una autoridad real (`Chain of trust NOT OK`), el escáner le baja la nota global a "T".

- Errores de nombre: Accedes por IP, pero el certificado es para `www.miserv.com`. Además, le falta el atributo SAN (Subject Alternative Name).

- Aviso de BREACH: Una advertencia menor porque el servidor Apache tiene activada la compresión HTTP (`gzip`).

3. Versión de TLS por defecto

La versión por defecto que intentará utilizar siempre el servidor es TLSv1.3.

Nota: En TLS 1.2 el algoritmo **RSA se utilizará como algoritmo de firma digital**, pero nunca como algoritmo para intercambiar claves.

— Hasta aquí tercer bloque: 3 p ---
